

PRE-TEST

1. **Antrean Pesanan:** Menyimpan nama pelanggan yang sedang antre secara berurutan.
2. **Meja Terisi:** Mencatat nomor meja yang sedang digunakan pelanggan. Sistem harus otomatis **menolak nomor ganda** jika kasir tidak sengaja memasukkan nomor meja yang sama dua kali.
3. **Daftar Harga:** Menyimpan harga dari masing-masing menu, di mana kasir bisa mencari harga berdasarkan nama menu.

Tugas Anda: Lengkapi kode di bawah ini dengan antarmuka (*Interface*) dan kelas implementasi (*Class*) Collection yang tepat.

```
import java.util.*;
```

```
public class PreTestKedaiKopi {
    public static void main(String[] args) {

        // TODO 1: Struktur data untuk menyimpan Antrean pelanggan.
        ____<String> antrean = new ____<>();

        // TODO 2: Struktur data untuk menyimpan Nomor Meja yang sedang dipakai.
        ____<Integer> mejaTerisi = new ____<>();

        // TODO 3: Struktur data untuk menyimpan Daftar Harga Menu.
        ____<String, Integer> hargaMenu = new ____<>();

        // --- SIMULASI (TIDAK PERLU DIUBAH) ---
        antrean.add("Budi");
        antrean.add("Siti");

        mejaTerisi.add(4);
        mejaTerisi.add(7);
        mejaTerisi.add(4); // Harus otomatis ditolak oleh sistem tanpa error

        hargaMenu.put("Kopi", 15000);
        hargaMenu.put("Teh", 10000);

        System.out.println("Daftar Antrean : " + antrean);
        System.out.println("Meja Terisi : " + mejaTerisi);
        System.out.println("Harga Kopi : Rp " + hargaMenu.get("Kopi"));
    }
}
```

Latihan

SOAL 1:

Buatlah class Buku yang memiliki atribut isbn (String) dan judul (String).

Gunakan Collections yang tepat untuk menyimpan katalog buku, di mana sistem harus bisa mencari objek Buku secara instan hanya dengan menggunakan kunci isbn. Masukkan minimal 3 objek buku ke dalam sistem.

SOAL 2:

Buatlah class Anggota yang memiliki atribut idAnggota (String), nama (String), dan tipe (String: "Mahasiswa" atau "Dosen"). Gunakan Collections yang **menolak duplikat**.

Anda wajib memastikan bahwa jika dua objek Anggota dibuat secara terpisah menggunakan new Anggota(...) namun memiliki idAnggota yang sama, sistem secara otomatis menolak objek kedua. (Clue: Anda harus meng-override metode tertentu di dalam class Anggota). Daftarkan 3 anggota valid dan 1 anggota duplikat untuk menguji sistem.

SOAL 3:

Sebuah antrean peminjaman dibuat. Jika yang meminjam adalah tipe "Dosen", maka ia memotong antrean dan diletakkan **paling depan**. Jika "Mahasiswa", masuk ke **paling belakang**.

Gunakan Collections yang mendukung manipulasi ujung antrean dengan cepat. Simpan data antrean dalam bentuk String dengan format "idAnggota#isbn". Buatlah 4 simulasi antrean campuran (Dosen dan Mahasiswa).

SOAL 4:

Proses antrean (Soal 3) dari urutan paling depan hingga habis. Pecah string untuk mendapatkan idAnggota dan isbn. (`.split("#");`)

Peminjaman dinyatakan berhasil HANYA JIKA:

1. idAnggota ada terdaftar di sistem (Soal 2).
2. isbn terdaftar di katalog buku (Soal 1).
3. Buku tersebut **belum/tidak sedang dipinjam** oleh orang lain.

Jika berhasil, catat data tersebut ke dalam sebuah Collections khusus "Buku Sedang Dipinjam".

Jika gagal, print alasan penolakan nya.

SOAL 5: (Bonus)

Dari data "Buku Sedang Dipinjam" (hasil Soal 4), cetaklah laporan ke layar. Laporan ini **wajib tercetak terurut secara alfabetis berdasarkan Judul Buku (A - Z)**.

SOAL POST-TEST

Sebuah bioskop independen membutuhkan program manajemen pemesanan tiket berbasis *Command Line Interface*(CLI). Anda diminta untuk merancang struktur data program ini menggunakan prinsip-prinsip *Java Collections Framework* (List, Set, dan Map).

Sistem ini harus beroperasi dengan ketentuan bisnis sebagai berikut:

1. Sistem harus menyimpan jadwal film hari ini. Bioskop memiliki 3 film yang tayang. Anda bebas menentukan judul dan harganya, tetapi sistem harus merelasikan sebuah "Kode Film" (misalnya: "F01") dengan "Objek Film" (yang berisi Judul dan Harga Tiket). Pencarian detail film oleh kasir akan dilakukan menggunakan Kode Film tersebut.

2. Studio bioskop tidak memiliki nomor kursi statis, melainkan pelanggan bebas memilih kode kursi (misal: "A1", "B4"). Sistem **wajib** menolak secara otomatis jika seorang pelanggan mencoba memesan kode kursi yang sudah dipesan oleh pelanggan sebelumnya untuk film yang sama. Anda harus memanfaatkan keunikan Set untuk menangani hal ini. (*Asumsi: Ujian ini hanya mensimulasikan 1 studio/1 film pada satu waktu antrean kasir*).

3. Setiap pemesanan tiket yang sukses (Kode Film valid dan Kursi tersedia) harus dicatat secara berurutan ke dalam sebuah riwayat transaksi. Setelah simulasi beberapa pesanan (minimal 3 pesanan sukses dan 1 pesanan gagal karena kursi ganda), program harus mencetak seluruh riwayat pesanan (berisi Nama Pemesan, Judul Film, Nomor Kursi, dan Harga) sesuai urutan transaksi dari yang paling awal hingga paling akhir.

Tugas Anda: Buatlah kode Java dari awal (termasuk pembuatan Class/Objek yang diperlukan) yang mensimulasikan proses pemesanan tiket di atas. Anda tidak perlu membuat menu interaktif dengan Scanner (input dari pengguna) jika tidak sempat; *hardcode* saja proses add(), put(), dan validasinya di dalam main method untuk menunjukkan bahwa logika Collections Anda berjalan dengan benar.